3. LoopFill Decoding

A looped language model applies the same block of layers T times to each token before predicting the next one. One pass of this block is a *loop step*. One loop step can advance many tokens by one loop each, even tokens at different loops. In greedy decoding, the next token is known only after the current one finishes all T loops, so every output token costs T loop steps. LoopFill starts this work early. It guesses upcoming tokens, called *drafts*, from text that appeared earlier and from the model's own predictions after a few loops, so no separate draft model is needed. Drafts only decide which work starts early. LoopFill emits only the greedy token predicted after all T loops at the last verified position.

Figure 2 follows the example of Figure 1, with $T = 3$ and one token per word. The prompt ends in `like`. While that position is still running, LoopFill can start work on several possible next tokens and their continuations. When `like` emits `to` at step 3, the matching branch carries its completed loops into the next step. The five-token output `to join Lean Club \n` takes five loop steps, compared with fifteen under greedy decoding.

Drafting from earlier text. As in SuffixDecoding (Oliaro et al., 2025), a *suffix tree* indexes the current prompt, the output verified so far, and earlier outputs. We match the last few tokens of a prefix against it and propose the tokens that followed the match. Drafts live in a *draft tree* whose root is the verified prefix x. A node u is x followed by draft tokens, and its child ua appends one token a. Running node u means processing its last token, so after T loops it yields the model's prediction of the token after u. In Figure 2(a), matching `like` in earlier text lets both `a` and `coffee` start in step 1 alongside the root. Each node attends to x and to its own ancestors, never to another branch.

Drafting from early loops. Each loop step advances every unfinished node by one loop. Children proposed during a step first run in the next one. When a node has run d_1 loops, and again at d_2 loops ($d_1 < d_2 < T$), we take its *shallow prediction* of the next token. We score the node's existing children together with the eight tokens this prediction ranks highest, which lets it add a child or prune an unlikely branch. Whenever it adds a child, we also query the suffix tree with the longer prefix, so a single shallow prediction can lead back into familiar text. In Figure 2(b), the first loop at `like` proposes `to`. A suffix match at `to` supplies `join Lean Hub`, so all four positions start in step 2. At the end of that step, shallow predictions at `join` and `Lean` add the children `the` and `Club`, respectively.

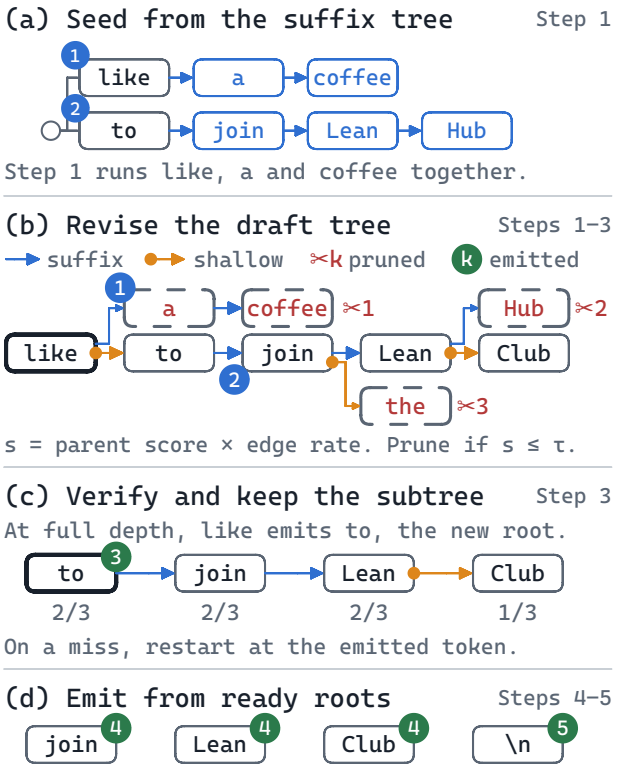


Figure 2. **LoopFill decoding on one example** ($T = 3$). (a) The numbered suffix matches supply continuations for `like` and `to`. (b) Blue arrows show suffix drafts. Orange arrows with dots show shallow drafts. Scissors mark the step when a branch is pruned. (c) When `to` becomes the root, fractions show the loops already completed at each node. (d) Green badges give emission steps. Three tokens are emitted in step 4.

Deciding which branches to compute. Every node occupies a slot in each loop step, and its work is wasted if any token on its path is wrong. We therefore score each edge with $\hat{q}(a \mid u)$, an estimate of the probability that a is the greedy token after u, given that u itself lies on the greedy path. We build these estimates as lookup tables before decoding. On calibration prompts, we follow the greedy output and tabulate how often proposed tokens with the same features match the greedy next token.

The available features grow as u runs more loops. Let m denote the suffix-match features: the match length, how often a followed the match, whether u has other children, and the depth of u in the draft tree. Let b indicate whether the suffix tree proposed a, and let r_i and p_i be the rank and probability of a in the shallow prediction after d_i loops. A token outside the top eight gets a reserved rank and $p_i = 0$. We update each edge estimate at node u using fixed match-rate tables M.

$$\hat{q}(a \mid u) \leftarrow \begin{cases} M_{\text{suf}}(m) & \text{on a suffix proposal,} \\ M_1(r_1, p_1, b) & \text{after } d_1 \text{ loops,} \\ M_2(r_1, r_2, p_2, b) & \text{after } d_2 \text{ loops.} \end{cases}$$

Each update replaces the previous estimate for that edge. The path score is $s(x) = 1$ at the root and $s(ua) = s(u)\,\hat{q}(a \mid u)$ below it. For example, if `to` scores 0.8 and `join` follows `to` at rate 0.5, the path `to join` scores 0.4. A node is kept only while its path score exceeds a threshold τ. Rescoring an edge changes every path score below it, and a node whose score falls to τ or below is pruned with its subtree. In Figure 2(b), rescoring removes the entire `a coffee` branch after step 1, `Hub` after step 2, and `the` after step 3. A higher τ wastes less work but may leave the correct next token out of the tree.

Verifying the draft. When the root finishes its T loops, its prediction gives the greedy token a, which we emit and add to the suffix tree. If xa is in the draft tree, it becomes the new root with score one. We keep its subtree with the work done there, discard its siblings, rescore the drafts below, and add suffix continuations. If the new root has also finished its T loops, we emit another token in the same loop step. In Figure 2(c), `to`, `join`, and `Lean` have each run two loops when `to` becomes the root. All three finish in step 4 and emit `join`, `Lean`, and `Club`, in that order. The position holding `Club` finishes in step 5 and emits the newline (Figure 2(d)). If xa is missing, we still emit a and start a new draft tree at xa. Either way, each emitted token is the greedy choice after the verified prefix, so LoopFill emits the same sequence as greedy decoding whenever both compute the same final predictions. Algorithm 1 summarizes one loop step.

Algorithm 1. LoopFill decoding

Start. Make the verified prefix x the root of a new draft tree. Add suffix drafts from x one token at a time while their path scores exceed τ.

Repeat for each loop step
- Advance every unfinished node by one loop.
- **While** the root x has finished T loops
  - Emit its greedy token a and add a to the suffix tree. Stop if the output is complete.
  - Set $x \leftarrow xa$. If x is missing from the draft tree, return to **Start**.
  - Make x the root with $s(x) = 1$. Keep its subtree and completed loops, recompute path scores from x, and add suffix drafts from x.
- **For each** surviving node u that has just run d_1 or d_2 loops
  - Score its children and the eight tokens its shallow prediction ranks highest. Keep ua if $s(ua) > \tau$, and otherwise prune it with its subtree.
  - Add suffix drafts from each newly added child.

References

Oliaro, G., Jia, Z., Campos, D., and Qiao, A. SuffixDecoding: Extreme speculative decoding for emerging AI applications. In *Advances in Neural Information Processing Systems*, 2025.